



中国科学院大学
University of Chinese Academy of Sciences



计算机体系结构铁人三项

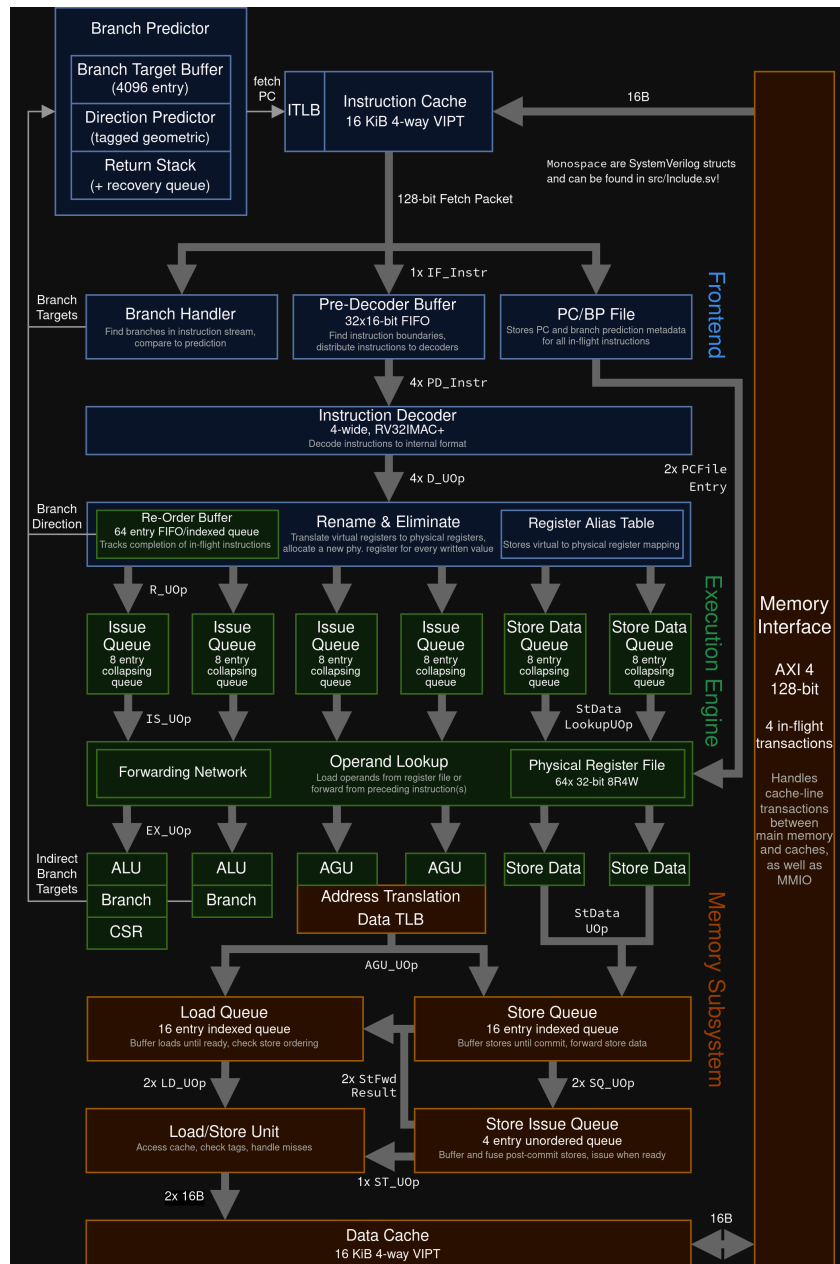
题 目：	SoomRV 设计文档
成 员：	刘卓敏 卫佳乐 陈冠宇 刘元昊 何国真
日 期：	2025.03.08

SoomRV 设计文档

1 项目背景分析

SoomRV 的目标是把四发射乱序执行、分支预测恢复、SV32 虚存、精确异常提交和 Linux 启动链路整合到同一实现中。工程难点不在单个模块，而在跨模块契约：

1. 前端必须在高吞吐下保持可恢复性，避免预测状态和 `fetch` 元数据错配。
2. 中后端必须实现“乱序执行 + 按序提交”的统一语义。
3. 访存路径要同时覆盖快路径（`cache hit`/前递）和慢路径（`miss/page walk/replay`）。
4. 构建链需可复现，支持回归与定向问题定位。



2 总体设计

2.1 系统分层与入口

这一层级划分的核心价值是把“计算语义”和“外设时序语义”解耦。`Core` 只表达架构层面的执行与提交，不直接承受 AXI 事务管理复杂度；`MemoryController` 则把并发请求转化为可追踪事务。

从工程经验看，这种边界一旦模糊，调试会迅速退化为跨模块猜测。下面这组代码片段对应的正是该边界最关键的连线点：顶层连接、Core 三路 memc 输出、SoC 收敛以及 memc 冲突仲裁入口。

```
// src/Top.sv:47-130
// Top 只负责把外部 AXI 模型和 SoC 接在一起。
ExternalAXISim extMem (...);
SoC soc (...);

// src/Core.sv:23-25
// Core 对外暴露三路 memc 请求：PC / LSU / BypassLSU。
assign OUT_memc[0] = PC_MC_if;
assign OUT_memc[1] = LSU_MC_if;
assign OUT_memc[2] = BLSU_MC_if;

// src/SoC.sv:66-79,124-144
// SoC 内部将 Core 请求交给 MemoryController，并回传状态。
MemoryController memc(
    .IN_ctrl(MemC_ctrl),
    .OUT_stat(MemC_stat),
    ...
);
Core core(
    ...
    .OUT_memc(MemC_ctrl),
    .IN_memc(MemC_stat)
);

// src/MemoryController.sv:135-161
// memc 在接收新请求时会检查 cache-line 冲突，避免同线并发 cache op。
always_comb begin
    OUT_stat.stall = {NUM_TFS_IN{1'b1}};
    selReq.cmd = MEMC_NONE;
    if (enqIdxValid) begin
        for (integer i = 0; i < NUM_TFS_IN; i=i+1) begin
            if (selReq.cmd == MEMC_NONE && IN_ctrl[i].cmd != MEMC_NONE) begin
                cacheAddrColl = 0;
                for (integer j = 0; j < `AXI_NUM_TRANS; j=j+1)
                    cacheAddrColl |= transfers[j].valid && ...;
                if (!cacheAddrColl) begin
                    selReq = IN_ctrl[i];
                    OUT_stat.stall = ~(1 << i);
                end
            end
        end
    end
end
```

2.2 数据面主链

IFetch -> InstrDecoder -> Rename -> IssueQueue -> Load(读操作数) -> 执行单元 -> Result/Flags
-> ROB

这条链路的设计重点不是“阶段越多越好”，而是让每一段只承担单一职责：解码只做语义展开，Rename 只建立依赖与序号，Issue 只做可发射决策，ROB 只做提交与精确状态。

这种分工使得性能优化和正确性验证可以分离推进，例如你可以单独调 Issue 选择策略，而不会直接破坏提交一致性。

```
// src/Core.sv:121-215
InstrDecoder idec(... .OUT_uop(DE_uop), .OUT_decBranch(decBranch));
Rename rn(... .IN_uop(DE_uop), .OUT_uop(RN_uop));
IssueQueue iq(... .IN_uop(RN_uop), .OUT_uop(IS_uop[i]));

// src/Core.sv:294-320
Load ld(
    .IN_uop(IS_uop),
    .OUT_uop(LD_uop)
);

// src/Core.sv:351-506
// 多执行单元并行，最后统一做 Result/Flags 拆分。
IntALU ialu(... .OUT_uop(resUOps[FU_INT]));
Divide div(...);
Multiply mul(...);
FPU fpu(...);
ResultFlagsSplit wbUOpSplit(wbUOp, flagUOps[i], resultUOps[i]);

// src/Core.sv:799-827
ROB rob(
    .IN_uop(RN_uop),
    .IN_flagUOps(flagUOps),
    .OUT_comUOp(comUOps),
    .OUT_bpUpdate(ROB_bpUpdate)
);
```

2.3 控制面反馈环

控制环路是 OoO 核稳定性的“隐性主干”。前端、提交端、访存端如果只做单向流水，系统在高压场景下会出现不可恢复的状态漂移。

因此 SoomRV 把“分支修正”“预测学习”“内存序回退”都显式建模成反馈信号，并在关键边界做统一收敛，避免多个局部恢复机制互相打架。

```
// src/Core.sv:49-64
// BranchSelector 汇聚多个分支来源，形成全局 branch。
BranchSelector bsel(
    .IN_branches(branchProvs[3:0]),
    .OUT_branch(branch)
);

// src/IFetch.sv:58-83
// IFetch 将执行/解码/前端修正合并为一个 BP_mispr。
always_comb begin
    BP_mispr = BH_fetchBranch;
```

```

    if (IN_branch.taken) BP_mispr = ...;
    else if (IN_decBranch.taken) BP_mispr = ...;
end

// src/ROB.sv:331-337
// 方向更新在提交侧产生，避免错误路径污染预测器。
if (deqFlags[i] == FLAGS_PRED_TAKEN || deqFlags[i] == FLAGS_PRED_NTAKEN) begin
    OUT_bpUpdate.valid <= 1;
    OUT_bpUpdate.branchTaken <= (deqFlags[i] == FLAGS_PRED_TAKEN);
end

// src/LoadBuffer.sv:318-330
// 内存序冲突触发 FLUSH_MEM_ORDER。
OUT_branch.taken = 1;
OUT_branch.cause = FLUSH_MEM_ORDER;
OUT_branch.tgtSpec = BR_TGT_NEXT;

```

3 关键数据结构与流水线契约

这里的结构体不是简单“字段集合”，而是模块间契约。每个阶段都基于这些字段做局部判断，如果字段语义不稳定，系统会出现很难定位的跨阶段错误。

下面第一段代码展示控制契约（`BranchProv`）和主链结构体族（`PD_Instr` 到 `ResultUOp`）；第二段再单独强调依赖标签与序号如何在同一结构里共存。

```

// src/Include.sv:450-466
// BranchProv 统一描述 flush/重定向语义与序号边界。
typedef struct packed {
    FlushCause cause;
    BranchTargetSpec tgtSpec;
    logic isSCFail;
    FetchOff_t fetchOffs;
    logic[31:0] dstPC;
    SqN sqN;
    SqN storeSqN;
    SqN loadSqN;
    logic flush;
    FetchID_t fetchID;
    logic taken;
} BranchProv;

// src/Include.sv:550-633
// 前端到发射：PD_Instr -> D_UOp -> R_UOp -> IS_UOp。
typedef struct packed { logic[31:0] instr; ... logic valid; } PD_Instr;
typedef struct packed { logic[31:0] imm; logic[4:0] rs1, rs2, rd; ... logic valid; } D_UOp;
typedef struct packed {
    logic availA; Tag tagA;
    logic availB; Tag tagB;
    logic availC; Tag tagC;
    SqN sqN; Tag tagDst;
    SqN storeSqN; SqN loadSqN;
    logic[NUM_PORTS_TOTAL-1:0] validIQ;
    logic valid;
} R_UOp;

```

```
typedef struct packed { logic[31:0] imm; Tag tagA, tagB, tagDst; SqN sqN; ... logic valid;
} IS_UOp;

// src/Include.sv:661-768
// 执行与访存: EX_UOp / AGU_UOp / FlagsUOp / ResultUOp。
typedef struct packed { logic[31:0] srcA, srcB, pc; ... SqN sqN; ... logic valid; } EX_UOp;
typedef struct packed { logic[31:0] addr; logic isStore, isLoad; ... SqN sqN, storeSqN,
loadSqN; logic valid; } AGU_UOp;
typedef struct packed { Tag tagDst; SqN sqN; Flags flags; logic doNotCommit; logic valid; }
FlagsUOp;
typedef struct packed { logic[31:0] result; Tag tagDst; logic doNotCommit; logic valid; }
ResultUOp;

// src/Include.sv:947-968
// 提交侧控制: Trap_UOp 与 BPUUpdate。
typedef struct packed { Flags flags; SqN sqN, loadSqN, storeSqN; FetchID_t fetchID; logic
valid; } Trap_UOp;
typedef struct packed { FetchOff_t fetchOffs; FetchID_t fetchID; logic branchTaken; logic
valid; } BPUUpdate;
```

这些结构体里最关键的三个字段组：

1. `sqN/loadSqN/storeSqN`：决定 flush/replay 的边界。
2. `tagA/tagB/tagC/tagDst`：决定依赖跟踪与写回可见性。
3. `fetchID/fetchOffs`：把提交事件精确映射回前端元数据。

这三类字段的组合，是 SoomRV 能同时处理“乱序执行”和“按序恢复”的根本原因。

实际调试时，定位问题通常不是看单个信号是否正确，而是检查这三组字段在多个阶段是否保持一致演化。

```
// src/Include.sv:592-607
// 依赖标签与顺序号在 R_UOp 中共存，这是 OoO 正确性的基础。
logic availA; Tag tagA;
logic availB; Tag tagB;
logic availC; Tag tagC;
SqN sqN;
Tag tagDst;
FetchID_t fetchID;
FetchOff_t fetchOffs;
SqN storeSqN;
SqN loadSqN;
```

4 详细实现

4.1 A：前端取指与分支预测

前端实现采用了典型但容易被误解的策略：目标预测（BTB/RET）与方向预测（TAGE）分离，恢复路径统一汇入 `BP_mispr`。

这套方案的优点是可维护性高，缺点是恢复窗口管理更苛刻，所以又引入了 `fetchLimit` 和多源 stall 条件来保护 PC/BP 元数据不被覆盖。

代码中可以看到这种权衡：预测器本体追求快路径，`IFetchPipeline/BranchHandler` 则承担异常边界修复。

```
// src/BranchPredictor.sv:96-139
// next-PC 选择：恢复优先，其次 BTB/RET，再退化为 dir-only。
```

```

always_comb begin
    OUT_predBr = '0;
    OUT_pc = pcReg;
    if (ignorePred) begin
        if (recovery.valid && recovery.tgtSpec != BR_TGT_MANUAL)
            OUT_pc = recoveredPC;
        end
    else if (BTB_br.valid && BTB_br.btype != BT_RETURN) begin
        OUT_predBr = BTB_br;
        OUT_predBr.taken |= TAGE_taken;
        if (OUT_predBr.taken) OUT_pc = OUT_predBr.dst;
    end
    else if (BTB_br.valid && BTB_br.btype == BT_RETURN && RET_br.valid) begin
        OUT_predBr = BTB_br;
        OUT_predBr.taken = 1;
        OUT_predBr.dst = OUT_curRetAddr;
        OUT_pc = OUT_predBr.dst;
    end
    else begin
        OUT_predBr.valid = 1;
        OUT_predBr.dirOnly = 1;
        OUT_predBr.taken = TAGE_taken;
    end
end

```

// src/BranchPredictor.sv:274-285
 // fetchLimit: 保护提交侧仍需读取的 fetchID 不被覆盖。

```

always_comb begin
    OUT_fetchLimit = FetchLimit'{valid: 0, default: 'x};
    if (bpUpdate.valid) begin
        OUT_fetchLimit.valid = 1;
        OUT_fetchLimit.fetchID = bpUpdate.fetchID;
    end
    else if (IN_bpUpdate.valid) begin
        OUT_fetchLimit.valid = 1;
        OUT_fetchLimit.fetchID = IN_bpUpdate.fetchID;
    end
end

```

// src/IFetch.sv:58-83
 // 多源重定向合流到 BP_mispr。

```

always_comb begin
    BP_mispr = BH_fetchBranch;
    if (IN_branch.taken) BP_mispr = ...;
    else if (IN_decBranch.taken) BP_mispr = ...;
end

```

// src/IFetchPipeline.sv:145-160
 // 前端停顿条件是多源叠加，不是单一 cache busy。

```

always_comb begin
    OUT_stall = 0;
    if (IN_pw.busy && IN_pw.rqID == RQ_ID) OUT_stall = 1;
    if (fetchLimit == (fetchID + FetchID_t'(fetch0.valid))) OUT_stall = 1;
    if (flushState != FLUSH_IDLE) OUT_stall = 1;
    if (IF_icache.busy) OUT_stall = 1;
end

```

```
// src/BranchHandler.sv:304-320
// predIllegal 时清理错误 BTB 项并要求重抓。
logic predIllegal = is32bit[IN_op.predBr.offts + ($bits(FetchOff_t)+1)'(1)];
btUpdate_c.valid = 1;
btUpdate_c.clean = 1;
decBranch_c.taken = 1;
decBranch_c.tgtSpec = BR_TGT_MANUAL;
```

4.2 B：解码、重命名、发射与读操作数

这一段的设计思想是“把冲突提前显式化”：不能发射的原因必须在 Rename/Issue 侧被明确编码，而不是把风险推迟到执行阶段再补救。

stall 条件并非单一开关，而是资源、顺序、数据可用性的组合谓词。这样虽然逻辑更复杂，但回放范围更小、问题定位更直接。

Load 采用“发射后读数”也是同一思路：把关键路径从发射前移走，代价是读操作数阶段更厚。

```
// src/InstrDecoder.sv:223-260
// 解码阶段已经处理 valid 门控和 fetchFault。
uop.valid = IN_instrs[i].valid && en && !decBranch.taken && !OUT_decBranch.taken;
if (IN_instrs[i].fetchFault != IF_FAULT_NONE) begin
    ... // 直接生成对应 trap 语义
end

// src/Rename.sv:80-117
// Rename stall: 端口反压 + tag 不足 + mispred 同拍保护。
OUT_stall = |portStall;
if (IN_mispredFlush && IN_uop[i].valid) OUT_stall = 1;
if ((!TB_tagsValid[i]) && IN_uop[i].valid && frontEn && TB_tagNeeded[i]) OUT_stall = 1;
RAT_issueValid[i] = !rst && !IN_branch.taken && frontEn && !OUT_stall && IN_uop[i].valid;

// src/Rename.sv:345-346
// load/store 序号采用不同增量语义，避免乱序边界歧义。
storeSqN: storeSqNs[i+1], // pre-increment
loadSqN: loadSqNs[i],     // post-increment

// src/IssueQueue.sv:176-204
// Issue 判定: ready + FU 限制 + CSR/SC 顺序 + LQ/SQ 窗口。
deqCandidate_c[i] = (i < insertIndex) &&
    &(queueAvail_c[0][i]) &&
    ...
    (!HasFU(FU_CSR) || queue[i].fu != FU_CSR || (i == 0 && queue[i].sqN == IN_commitSqN))
&&
    ...
    (queue[i].opcode < LSU_SC_W || $signed(queue[i].storeSqN - IN_maxStoreSqN) <= 0);

// src/IssueQueue.sv:303-307
// 慢单元写回预约，减少写回口冲突。
case (deqEntry.fu)
    FU_DIV: reservedWBs <= ... | (1 << (IDIV_DLY - 1));
    FU_MUL: reservedWBs <= ... | (1 << (IMUL_DLY - 1));
endcase

// src/Load.sv:37-122
// 读操作数来源: forward -> RF -> PCFile。
```



```

forwards[i+NUM_ZC_FWDS].valid = IN_resultUOps[i].valid &&
!IN_resultUOps[i].tagDst[$bits(Tag)-1];
match[i][j] = (forwards[j].valid && forwards[j].tag == lookups[i]);
OUT_rfReadReq[i].valid = IN_uop[i].valid && !IN_uop[i].tagA[$bits(Tag)-1];
OUT_pcRead[i].addr = IN_uop[i].fetchID;
OUT_uop[i].pc = {IN_pcReadData[i].pc[30:$bits(FetchOff_t)], outUOpReg[i].fetchOffs, 1'b0};

```

4.3 C：执行单元、提交与异常控制

执行与提交的分工在这里非常清晰：执行单元负责尽快给出结果，ROB 决定哪些结果可以成为架构状态。

把 `BPUpdate` 放在提交侧是一个典型“正确性优先于时效”的选择：更新慢一拍可以接受，但错误路径训练预测器会长期放大代价。

Trap/CSR 路径的实现也遵循同样原则，所有会改变全局控制流的事件都通过统一出口处理，避免并发控制源造成的竞态。

```

// src/Core.sv:351-506
// 执行单元并行 + 统一写回拆分。
IntALU ialu(...);
Divide div(...);
Multiply mul(...);
FPU fpu(...);
ResultFlagsSplit wbUOpSplit(wbUOp, flagUOps[i], resultUOps[i]);

// src/IntALU.sv:206-243
// 分支错误时生成 flush; 间接跳转还会产出 BTUpdate。
if (isBranch) begin
    if (branchTaken != IN_uop.bpi.taken && IN_uop.opcode != BR_JAL) begin
        branch_c.taken = 1;
        branch_c.cause = branchTaken ? FLUSH_BRANCH_TK : FLUSH_BRANCH_NT;
    end
end else if (IN_uop.opcode == BR_V_RET || IN_uop.opcode == BR_V_JALR || IN_uop.opcode ==
BR_V_JR) begin
    if (!indBranchCorrect || !IN_uop.bpi.taken) begin
        branch_c.taken = 1;
        btUpdate_c.valid = 1;
    end
end
end

// src/ROB.sv:282-337
// 提交门控 + 提交侧预测更新。
reg isRenamed = ...;
reg isExecuted = deqFlags[i] != FLAGS_NX;
reg noFlagConflict = (!pred || (deqFlags[i] == FLAGS_NONE));
reg lbAllowsCommit = ...;
reg sqAllowsCommit = ...;
if (!temp && isRenamed && ((isExecuted && noFlagConflict && sqAllowsCommit &&
lbAllowsCommit) || timeoutCommit)) begin
    ...
    if (deqFlags[i] == FLAGS_PRED_TAKEN || deqFlags[i] == FLAGS_PRED_NTAKEN) begin
        OUT_bpUpdate.valid <= 1;
        OUT_bpUpdate.branchTaken <= (deqFlags[i] == FLAGS_PRED_TAKEN);
    end
end
end

// src/TrapHandler.sv:101-238

```

```

// ordering 类事件与 trap/interrupt 在这里统一收口。
if (IN_trapInstr.flags == FLAGS_FENCE || IN_trapInstr.flags == FLAGS_ORDERING ||
IN_trapInstr.flags == FLAGS_XRET || ...) begin
    OUT_branch_c.taken = 1;
    OUT_branch_c.flush = 1;
    OUT_branch_c.cause = FLUSH_ORDERING;
end
else if (IN_trapInstr.timeout || (IN_trapInstr.flags >= FLAGS_ILLEGAL_INSTR &&
IN_trapInstr.flags <= FLAGS_ST_PF)) begin
    trapInfo_c.valid = 1;
    OUT_branch_c.taken = 1;
    OUT_branch_c.tgtSpec = BR_TGT_MANUAL;
end

// src/CSR.sv:403-456,267,595-596
// CSR 对外输出 trapControl/dec/vmem; tinfo 实现为只读 0。
assign OUT_trapControl.interruptPending = interrupt;
assign OUT_dec.allowWFI = (priv == PRIV_MACHINE) || (priv == PRIV_SUPERVISOR &&
!mstatus.tw);
always_comb begin
    vmem_c.rootPPN = satp.ppn;
    vmem_c.sv32en_ifetch = satp.mode && priv != PRIV_MACHINE;
end
CSR_tinfo = 12'h7A4; // trigger module info
... CSR_tinfo, ... : rdata = 0;

```

4.4 D：内存子系统与 MMU

D 段的关键不是单点性能，而是“快路径高效、慢路径可恢复”。

SoomRV 的做法是：AGU 尽早分类请求，LoadBuffer/SQ/SQB 处理前递与重放边界，LSU 统一 cache/MMIO/BLSU 仲裁，TLB miss 则通过 TMQ+PageWalker 形成闭环。

这套路径里最重要的工程约束是回放可解释性：每一次 nack/flush 都必须能映射回明确序号边界，否则 Linux 场景下会出现间歇性错误而难以复现。

```

// src/AGU.sv:50-195
// AGU 统一生成 load/store/atomic/cache-op 属性。
aguUOp_c.addr = addr;
aguUOp_c.sqN = IN_uop.sqN;
aguUOp_c.storeSqN = IN_uop.storeSqN;
aguUOp_c.loadSqN = IN_uop.loadSqN;
if (IN_uop.opcode < LSU_SC_W) begin
    aguUOp_c.isLoad = 1;
    ...
end else begin
    aguUOp_c.isStore = 1;
    ...
end

// src/AGU.sv:335-433
// TLB miss 进入 TMQ; 无异常时发出 AGU_UOp。
if (issUOp_c.valid && IN_vmem.sv32en && exceptFlags == FLAGS_NONE && !IN_tlb.hit) begin
    tlbMiss = 1;
    TMQ_enqueue = 1;
end
...
if (doIssue) begin

```

```

    if (((isLoad || isAtomic) && exceptFlags != FLAGS_NONE) || isStore) begin
        OUT_uop <= issResUOp_c; // 先给 ROB flags
    end
    if (exceptFlags == FLAGS_NONE || exceptFlags == FLAGS_ORDERING) begin
        OUT_aguOp <= issUOp_c; // 真正送 LSU
    end
end

// src/TLBMissQueue.sv:79-116
// PageWalk 返回后按 VPN 匹配挂起请求并重新出队。
if (IN_pw.valid) begin
    if (queue[i].valid && (... vpn match ...)) ready[i] <= 1;
end
if ((!OUT_uop.valid || IN_dequeue) && idxOutValid) begin
    OUT_uop <= queue[idxOut];
end

// src/PageWalker.sv:26-31,96-151
// PageWalker 使用 doNotCommit + TAG_ZERO 识别自己的 load 结果并支持重发。
if (IN_ldResUOp[i].valid && IN_ldResUOp[i].doNotCommit && IN_ldResUOp[i].tagDst ==
TAG_ZERO)
    pwLdRes = IN_ldResUOp[i];
...
if (IN_ldAck[i].valid && IN_ldAck[i].external && IN_ldAck[i].fail)
    OUT_ldUOp.valid <= 1; // 外部失败重发

// src/LoadBuffer.sv:84-120,318-330
// 早发失败/冲突转晚发; 内存序冲突触发 FLUSH_MEM_ORDER。
delayLoad[h] = nonSpeculative[h] || IN_uop[h].earlyLoadFailed;
...
if (storeIsConflict[i]) begin
    OUT_branch.taken = 1;
    OUT_branch.cause = FLUSH_MEM_ORDER;
end

// src/StoreQueue.sv:53-61
// SQ 仅允许提交边界内条目出队。
RangeMaskGen readyRangeGen(
    .IN_startIdx(baseIndex[IDX_LEN-1:0]),
    .IN_endIdx(IN_comStSqN[IDX_LEN-1:0]),
    .OUT_range(entryReady_c)
);

// src/StoreQueueBackend.sv:85-106,222-264
// SQB 融合同 line store; nonce 区分重发轮次。
if (IN_uop[i].addr[31:AXI_BWIDTH_E] == fusedUOp_c.addr[29:AXI_BWIDTH_E-2]) begin
    fusedUOp_c.data[...] = IN_uop[i].data[...];
    fusedUOp_c.wmask[j] = 1;
end
...
if (stAck_r.valid && stAck_r.nonce == evicted[stAck_r.idx].nonce) begin
    if (!stAck_r.fail) evicted[stAck_r.idx].valid <= 0;
end

// src/LoadStoreUnit.sv:108-128,499-510,677-686
// LSU 统一 cache/MMIO/BLSU 路径, 并用 nack 触发 replay。

```

```

BypassLSU bypassLSU(...);
...
if (!isMMIO && stFwd[i].conflict) begin
    miss[i].mtype = CONFLICT;
    miss[i].valid = 1;
end
...
else if (curLd[i].valid) begin
    OUT_ldAck[i].valid = 1;
    OUT_ldAck[i].fail = 1;
end

// src/TLB_fixed.sv:90-114 vs src/TLB.sv:82-103
// fixed 版新增 already_exists, 原版无查重。
logic already_exists;
already_exists = 1'b0;
for (integer j = 0; j < ASSOC; j=j+1)
    if (tlb[idx][j].valid && tlb[idx][j].vpn == IN_pw.vpn[19:$clog2(LEN)] && tlb[idx]
[j].isSuper == IN_pw.isSuperPage)
        already_exists = 1'b1;
if (!already_exists) begin
    tlb[idx][assocIdx].vpn <= IN_pw.vpn[19:$clog2(LEN)];
    ...
end

```

4.5 E：全局串联与系统集成

E 段的价值在于把 A-D 的局部机制放到系统尺度解释：谁负责功能语义，谁负责带宽仲裁，谁负责状态可观测。代码里可以看到 [Top](#) 的极简职责、[SoC](#) 的接口拼接职责，以及 [MemoryController](#) 的事务化职责。这种分层使得系统调试可以先判定“问题在核内还是在内存接口”，而不必从一开始就全局搜索。

```

// src/Top.sv:11-14,47-130
// Top 的 halt 定义为 poweroff/reboot, 便于 testbench 判停。
assign OUT_halt = SOC_poweroff || SOC_reboot;
ExternalAXISim extMem(...);
SoC soc(...);

// src/SoC.sv:64-79,193-225,251-316
// SoC 把 Core 接口映射到本地 cache/table 和 memc。
MemoryController memc(...);
CacheArbiter dcacheArb(...);
MemRTL dcache(...);
MemRTL1RW dhtable(...);
MemRTL1RW ictable(...);

// src/MemoryController.sv:147-154,636-697
// memc 既避免 cache-line 冲突, 也提供 refill 前向转发。
cacheAddrColl |= transfers[j].valid && ...;
...
case (selReq.cmd)
    MEMC_REPLACE: begin
        transfers[enqIdx].needReadRq <= '1;
        transfers[enqIdx].needWriteRq <= 2'b11;
    end
endcase
...

```

```
ldDataFwd.data <= s_axi_rdata;
ldDataFwd.addr <= {...};
ldDataFwd.valid <= 1;
```

5 运行方法

5.1 环境与构建开关

构建配置直接决定实验可复现性。本项目把两类高频实验开关放在同一层：`TLB_IMPL` 用于行为对比，`HARDFLOAT_DIR` 用于外部接入验证。

这种做法避免了“改源码切换实验模式”的高风险流程，尤其适合团队协作阶段的并行验证。

```
# Makefile
COSIM ?= 1
HARDFLOAT_DIR ?= hardfloat
HARDFLOAT_SRC := $(wildcard $(HARDFLOAT_DIR)/*.v)
TLB_IMPL ?= fixed
ifeq ($(TLB_IMPL),orig)
TLB_SRC := src/TLB.sv
else ifeq ($(TLB_IMPL),fixed)
TLB_SRC := src/TLB_fixed.sv
endif
VERILATOR_CFG = ... -I$(HARDFLOAT_DIR)
SRC_FILES = ... $(TLB_SRC) ... $(HARDFLOAT_SRC)

setup:
    git submodule update --init --recursive
    cd riscv-isa-sim && ./configure ...
    make -j $(nproc) -C riscv-isa-sim
```

常用命令：

下面这些命令按“依赖初始化 -> 默认构建 -> 变体构建”排序，适合新环境首次复现实验。

```
cd /home/zoomin/codes/RISCV/SoomRV
make setup
make soomrv
make soomrv TLB_IMPL=orig
make soomrv TLB_IMPL=fixed
make soomrv HARDFLOAT_DIR=/abs/path/to/hardfloat
```

5.2 Linux 镜像构建与运行

Linux 构建链路采用“Buildroot 产核 + OpenSBI 拼装”的两段式流程。核心考虑是把平台固件与内核镜像职责分离，便于单独定位启动问题。

镜像拼接阶段明确做了 4MiB 对齐，这一步不是格式美观问题，而是 OpenSBI 启动约束，省略会直接导致运行异常。

```
# 顶层 Makefile
linux: soomrv
    make -C test_programs/linux
    ./obj_dir/VTop --device-tree=test_programs/linux/device_tree.dtb --backup-
file=soomrv.backup test_programs/linux/linux_image.elf
```

```
# test_programs/linux/Makefile
$(BUILDROOT_DIR)/output/images/Image: ...
    make -C buildroot

linux_image.bin: $(BUILDROOT_DIR)/output/images/Image
    cp $(BUILDROOT_DIR)/output/images/fw_jump.bin linux_image.bin
    dd if=/dev/zero bs=1 seek=4194304 count=0 of=linux_image.bin
    cat $(BUILDROOT_DIR)/output/images/Image >> linux_image.bin

linux_image.elf: linux_image.bin
    $(RV32_OBJS) ... --output-target=elf32-little linux_image.bin linux_image.elf
```

5.3 ISA 回归脚本

ISA 回归脚本的意义在于把“单测可过”提升为“批量集可过”。脚本按类别批量运行，并在失败时追加 `-x 0` 的调试输出，减少二次复现成本。

此外脚本支持命令行覆盖测试目录，适合做子集回归或增量验证。

```
# scripts/test_suite.py
test_dir = "riscv-tests/isa"
if len(sys.argv) > 1:
    test_dir = sys.argv[1]

binary = "./obj_dir/VTop"
for category in categories:
    tests = [test for test in arr if test.find(category) != -1]
    for test in tests:
        result = os.popen(f"{binary} -t {test}").read()
        if not result.startswith("PASSED"):
            print(os.popen(f"{binary} -x 0 -t {test} 2>&1 | tail -n32").read())
```

运行：

命令如下，默认目录是 `riscv-tests/isa`：

```
python scripts/test_suite.py
```

6 测试方法与结果

6.1 测试方案总览

类别	命令	目标	日志
Linux 启动验证（旧镜像）	./obj_dir/VTop ... linux.bak/linux_image.elf	验证 S 态、MMU、用户态初始化	docs/logs/linux_boot_legacy.log
Linux 启动验证（新镜像）	./obj_dir/VTop ... linux/linux_image.elf	验证 Buildroot 新链路可运行性	docs/logs/linux_boot_current.log
Linux 性能采样（旧）	./obj_dir/VTop --perfc ... linux.bak	IPC/MPKI 区间统计	docs/logs/linux_perfc_legacy.log
Linux 性能采样（新）	./obj_dir/VTop --perfc ... linux	IPC/MPKI 区间统计	docs/logs/linux_perfc_current.log

类别	命令	目标	日志
裸机程序回归	<code>./obj_dir/VTop --perfc test_programs/*.s</code>	功能与性能 smoke	<code>docs/logs/baremetal_perfc_smoke.log</code>
开发微测试 (TLB 修复)	<code>make -C test_programs/dev compare</code>	复现并验证重复插入修复	<code>docs/logs/dev_tlb_compare.log</code>
ISA 回归脚本	<code>python scripts/test_suite.py</code>	rv32* 指令集批量回归	本次未提供日志

6.2 Linux 启动结果

6.2.1 旧镜像 (`docs/logs/linux_boot_legacy.log`)

1. OpenSBI 正常: `OpenSBI v1.2` (第 7 行)。
2. 内核启动: `Linux version 6.3.4 ... #4 Wed Nov 22 21:12:23 CET 2023` (第 59 行)。
3. 进入用户态: `Run /init as init process` (第 131 行)。
4. Buildroot 登录成功: `Welcome to SoomRV Buildroot` (第 137 行)。

6.2.2 新镜像 (`docs/logs/linux_boot_current.log`)

1. OpenSBI 正常: `OpenSBI v1.2` (第 3 行)。
2. 内核启动: `Linux version 6.19.5 ... #2 Fri Mar 6 23:00:43 CST 2026` (第 55 行)。
3. 进入用户态: `Run /init as init process` (第 132 行)。
4. 用户态告警: `IRQ index 0 not found` (第 118 行)、`/sbin/ifup: not found` (第 137 行)。
5. 仍出现登录提示: `Welcome to Buildroot` (第 141 行)。

结论: CPU 主链路可稳定启动 Linux; 新镜像问题集中在 rootfs/network 侧配置。

6.3 Linux `--perfc` 统计

统计口径: 使用 Python 脚本扫描日志中每条 `instret ... # <IPC> IPC`、`mispredicts ... # <MPKI> MPKI` 与 `branch mispredicts ... # <RATE>%` 记录, 计算样本数、均值、最小值、最大值。

日志	IPC 样本数	平均 IPC	IPC 区间	MPKI 样本数	平均 MPKI	MPKI 区间
<code>docs/logs/linux_perfc_legacy.log</code>	43	1.370866	1.097686 ~ 2.333168	43	16.013032	1.552224 ~ 23.971078
<code>docs/logs/linux_perfc_current.log</code>	46	1.159559	0.802979 ~ 2.111755	46	15.609377	2.621054 ~ 21.926403

指标说明:

1. 平均 IPC: 统计窗口内 IPC 样本的算术平均值, 反映整体指令吞吐能力。
2. IPC 区间: IPC 样本的最小值到最大值, 反映运行阶段的吞吐波动范围。
3. MPKI 样本数: 纳入统计的 MPKI 记录数量 (MPKI 为每千条指令的分支误判数)。
4. 平均 MPKI: MPKI 样本的算术平均值, 数值越低代表分支预测整体越稳定。
5. MPKI 区间: MPKI 样本的最小值到最大值, 反映分支误判压力的波动范围。

补充: 分支误判率均值分别为 8.099647% (legacy) 和 7.660898% (current)。

6.4 裸机程序结果 ([docs/logs/baremetal_perfc_smoke.log](#))

程序	cycles	IPC	MPKI	branch mispredict rate
dhry_1.s	294438	2.121554	0.156884	0.085696%
add.s	123	0.365854	0.000000	0.000000%
atomic2.s	7862	1.308446	0.874891	0.584226%
atomic.s	7250	0.219448	279.698303	6.015038%
bf.s	7527	1.204464	25.369512	8.702349%
bitmanip.s	1322	0.620272	0.000000	0.000000%
memcpy.s	8349	3.438496	0.139334	0.097537%

6.5 TLB 修复对比 ([docs/logs/dev_tlb_compare.log](#))

修复动机：原始 `TLB.sv` 在 page-walk 回填路径上，满足 `IN_pw.valid` 后会直接写入 `tlb[idx][assocIdx]`，但没有检查“同一 set 是否已存在相同 VPN 条目”。这会导致同一页表映射被重复插入（源码内也有 `FIXME ... might double insert ...` 注释）。

修复前的具体问题：

1. 同一 VPN 在同一 set 多路重复驻留，等效降低了 TLB 可用相联度。
2. 重复回填会推进替换计数器，挤占本应保留给其他 VPN 的 way，增加后续冲突 miss 风险。
3. 在高 miss 或频繁 page-walk 场景下，TLB 结构状态更容易抖动，带来不必要的性能波动。

本次修复：`TLB_fixed.sv` 增加 `already_exists` 判定，仅在未命中已有同 VPN 条目时才执行插入。

对比结果 ([docs/logs/dev_tlb_compare.log](#))：

1. 原版：`RESULT count=2`，`RESULT_DUPLICATE=1`。
2. 修复版：`RESULT count=1`，`RESULT_DUPLICATE=0`。
3. 自动判定：`PASS: fixed TLB prevents duplicate insertion`。

这里引用的测试点在 `test_programs/dev/tlb_dup_tb.sv:69-96`。

这个测试采用白盒计数而不是仅看 hit/miss，是因为问题本质是“同 VPN 在同 set 内重复驻留”，外部行为在短窗口内可能并不立即暴露。

通过直接读取 `dut.tlb`，可以把修复效果从“推测成立”升级为“结构状态已验证”。

```
// test_programs/dev/tlb_dup_tb.sv:69-96
// 构造同 VPN 二次回填，统计 set 内重复条目数。
pw.valid = 1;
pw.vpn = vpn;
...
@(posedge clk);
pw.valid = 0;
...
for (int j = 0; j < ASSOC; j++) begin
    if (dut.tlb[idx][j].valid && dut.tlb[idx][j].vpn == vpn[19:$clog2(SIZE / ASSOC)] &&
        dut.tlb[idx][j].isSuper == 0)
        count++;
end
if (count > 1) $display("RESULT_DUPLICATE=1");
else $display("RESULT_DUPLICATE=0");
```

7 成员分工与贡献

卫佳乐

1. 负责范围：前端取指、预测恢复、BTB/TAGE/ReturnStack。
2. 贡献点：统一前端改向语义，补齐 `fetchLimit` 风险分析，梳理 `predIllegal` 修复路径。

刘元昊

1. 负责范围：Decode/Rename/Issue/Load 读操作数链。
2. 贡献点：明确 tag/sqN 契约、IssueQueue 发射条件与写回冲突处理。

何国真

1. 负责范围：执行单元、ROB 提交、CSR/Trap 控制。
2. 贡献点：明确提交门控与提交侧预测更新策略，完成 CSR 兼容性收敛。

陈冠宇

1. 负责范围：AGU/LSU/SQ/SQB/TLB/PageWalker。
2. 贡献点：梳理访存快慢路径、TLB miss 协同、内存序回放机制。

刘卓敏

1. 负责范围：Top/SoC/Core/memc 等各个模块的串联，其他改进包括 TLB 修复、新版本 Linux 构建测试、工具链更新与兼容性修复等。
2. 贡献点：统一 A-D 接口口径，完成系统级数据面/控制面。

8 部分额外改进说明

1. TLB 修复：新增 `TLB_fixed` 查重插入，并通过 `test_programs/dev` 对比测试验证通过。
2. Linux 启动链路排障：定位新镜像问题主要在用户态配置而非核心流水线崩溃。
3. HardFloat 外部接入：顶层 Makefile 支持 `HARDFLOAT_DIR`，并补充 `hardfloat_ext_tb.sv`。
4. 生成头兼容层：`sim/sc_stub.hpp` 增加 `sc_dt::sc_bv` 别名，解决生成头命名空间兼容问题。

9 总结与展望

1. SoomRV 已形成可运行的四发射 OoO 主链，能够稳定进入 Linux 用户态。
2. 内存子系统具备前递、回放、TLB miss/page-walk 协同与异常收敛机制。
3. TLB 重复插入问题已有“复现-修复-验证”闭环。
4. 构建侧支持 TLB 实现切换和 HardFloat 外接，便于后续实验与回归。